

Contributions schema

Eight tables derived field by field from the eight-step Add Contribution wizard. Nothing here is built yet. This is the shape to agree on before writing entities and the migration.

8 new tables	11 foreign keys	3 JSON columns	1 unique index	6 new enums	0 columns added to existing tables
------------------------	---------------------------	--------------------------	--------------------------	-----------------------	---

Changed in v1.1. The first draft had twelve tables. Three of them held nothing but a list of values, so they became JSON columns on the root row using EF Core 9 primitive collections, and a fourth was folded into `ContributionLinks`. Section 3 has the evidence this works.

Where each wizard step lands

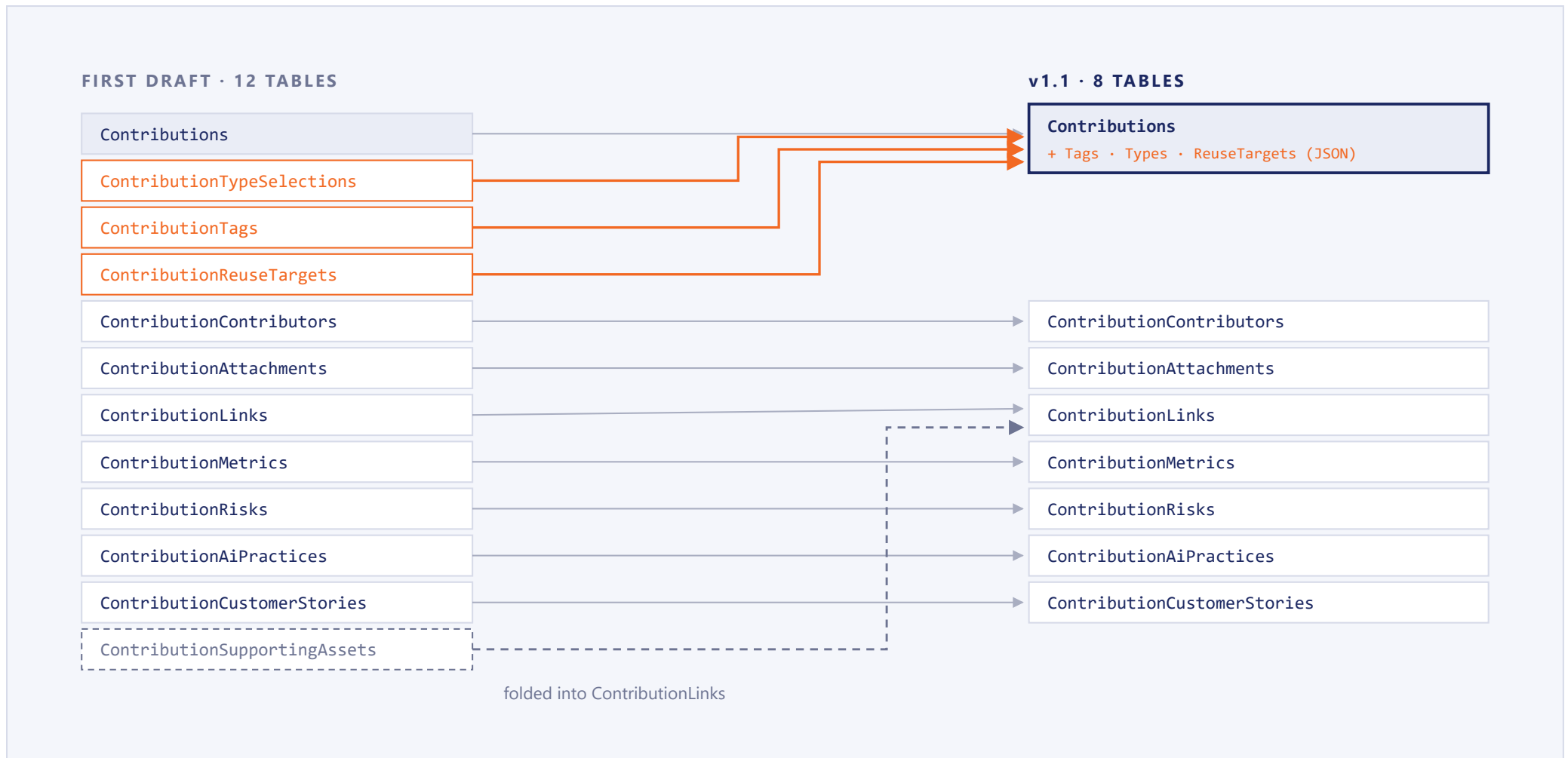
The wizard collects one flat object on the client (the `Contribution` interface in `ContributionContext.tsx`). Four of the eight steps now write into a single row.

STEP	SCREEN	LANDS IN	SHAPE
1	Initiative	<code>Contributions.InitiativeId</code>	FK
2	Type	<code>Contributions.Types</code>	JSON array
3	Summary	<code>Contributions.Title</code> , <code>.Description</code> , <code>.KeyTakeaway</code> , <code>.Priority</code> , <code>.Tags</code>	columns + JSON array
4	Details	<code>ContributionMetrics</code> · <code>Risks</code> · <code>AiPractices</code> · <code>CustomerStories</code>	0:1 table each
5	Evidence	<code>ContributionAttachments</code> , <code>ContributionLinks</code>	1:n each
6	Contributors	<code>ContributionContributors</code>	1:n
7	Visibility	<code>Contributions.ReuseTargets</code>	JSON array
8	Review	<code>Contributions.Status</code>	column

Step 4 still shapes the design. Its sections appear only when the matching type is selected in step 2, so each stays an optional 1:1 table rather than a block of nullable columns on the root row. A Progress Update physically cannot carry a risk severity.

What collapsed, and what resisted

The dividing line is whether a row carries anything of its own beyond the value. A bare list of values does not need a table. A row with a payload, a foreign key, or its own endpoint does.



— became a JSON column on the root row - - - folded into another table

The gaps on the right are the point. **Tags**, **Types**, and **ReuseTargets** were tables whose only columns were an id, a parent id, and one value. That is exactly what EF Core 9 primitive collections are for. **SupportingAssets** collected a link and a description, which **ContributionLinks** already stores.

What resisted, and why

- `ContributionContributors` carries `ResponsibilityArea` and `IsPrimary` per row, and a foreign key to `Users` . JSON cannot express a foreign key, and "contributions by person" drives the Team Contributions page.
- `ContributionAttachments` needs a row id for the download and delete endpoints, and a blob lifecycle to match.
- `ContributionLinks` holds objects, not primitives, so primitive collections do not apply.
- The four detail tables hold the analytic payload: filtering risks by severity, summing time saved, computing metric deltas, pulling quotes for Testimonials. Buried in a blob, every one of those becomes an `OPENJSON` exercise. `ContributionRisks` also has a foreign key to `Users` .

Evidence the JSON columns work

Verified against EF Core 9.0.0 on net9.0, the versions in `TeamIntelligenceHub.Infrastructure.csproj` , in a throwaway project rather than from memory.

model mapping				one table, no shadow entity
PROPERTY	CLR	COLUMN	PRIMITIVE COLLECTION	
Id	Int32	int	False	
Title	String	nvarchar(200)	False	
Tags	List<string>	nvarchar(1000)	True	
Types	List<enum>	nvarchar(200)	True, stores member names	

The queries still translate to SQL

None of these fall back to client evaluation. All three run on the server.

filter by one tag

LINQ: `c => c.Tags.Contains("APAC")`

```
WHERE N'APAC' IN (
    SELECT [t].[value]
    FROM OPENJSON([c].[Tags]) WITH ([value] nvarchar(max) '$') AS [t]
)
```

tag vocabulary for the typeahead

LINQ: `SelectMany(c => c.Tags).Distinct()`

```
SELECT DISTINCT [t].[value]
FROM [Contributions] AS [c]
CROSS APPLY OPENJSON([c].[Tags]) WITH ([value] nvarchar(max) '$') AS [t]
```

tag facet counts

LINQ: `SelectMany().GroupBy().Count()`

```
SELECT [t].[value] AS [Tag], COUNT(*) AS [Count]
FROM [Contributions] AS [c]
CROSS APPLY OPENJSON([c].[Tags]) WITH ([value] nvarchar(max) '$') AS [t]
GROUP BY [t].[value]
```

Edit works by mutating the list

The concern with a JSON column is whether EF notices a tag added or removed in place, or only on reassignment. It notices.

```
after attach          : Unchanged
after Tags.Add(...)   : Modified   TagsModified=True
after Tags.Remove(...): Modified   value=APAC,Pilot
value comparer        : ListOfReferenceTypesComparer
deep-compares lists   : True
```

So edit is `contribution.Tags.Remove("FY26")` then `SaveChangesAsync()`. No delete-and-reinsert, which is what `ActivityRepository.ReplaceMentionsAsync` has to do today for a child table.

Configuration

```
builder.PrimitiveCollection(c => c.Tags).HasMaxLength(1000);

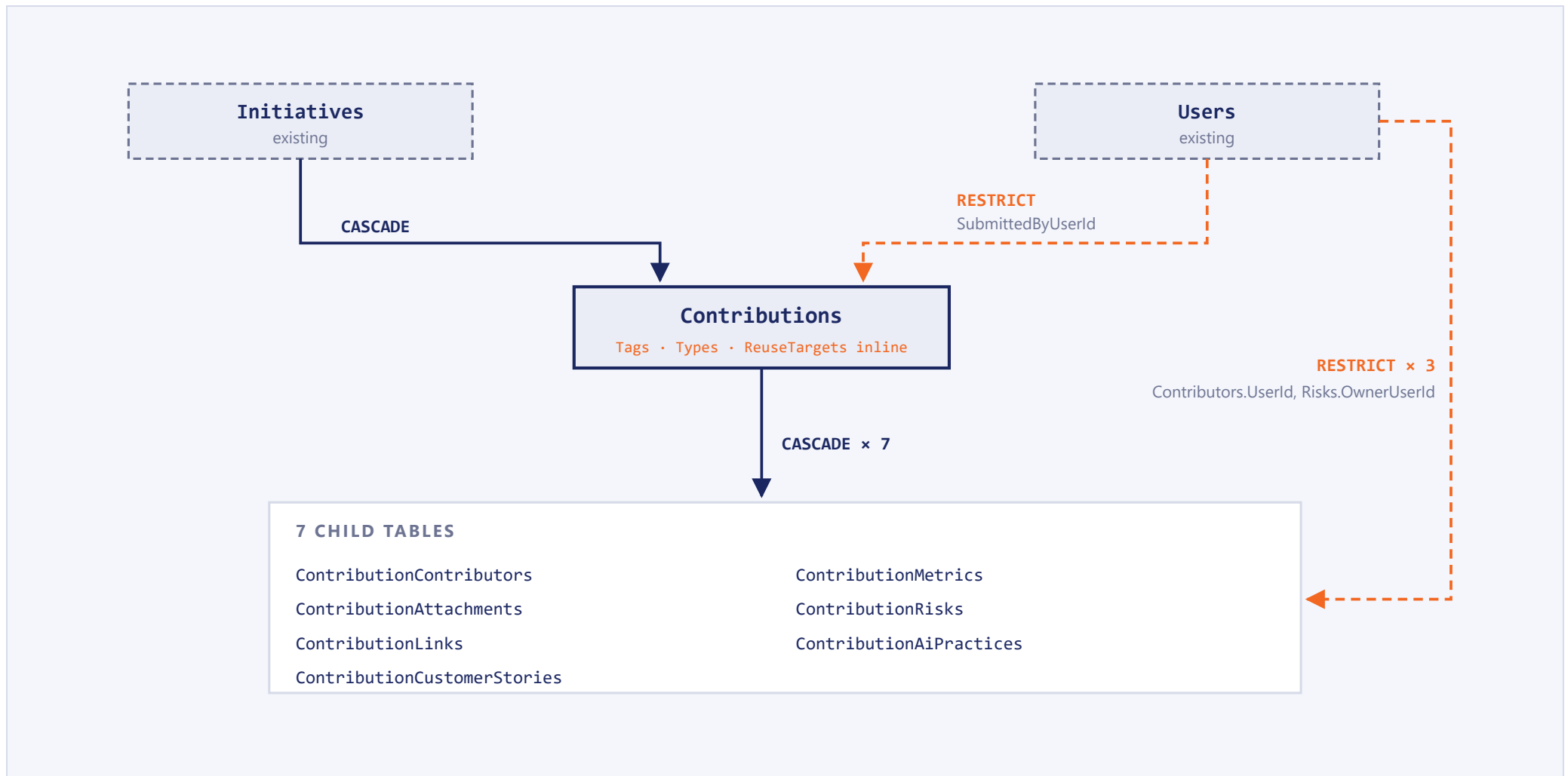
builder.PrimitiveCollection(c => c.Types)
    .HasMaxLength(200)
    .ElementType(e => e.HasConversion<string>());

builder.PrimitiveCollection(c => c.ReuseTargets)
    .HasMaxLength(400)
    .ElementType(e => e.HasConversion<string>());
```

Set HasMaxLength on all three. Without it EF emits `nvarchar(max)`, which is stored off-row and reads slower.

What a delete takes with it

Two existing tables anchor the new ones. SQL Server permits only one cascade path into any table, and the app already relies on that budget for Initiatives, so Users gets the restricting edges.



— CASCADE - - - RESTRICT, the delete is refused

Deleting an Initiative removes its Contributions and everything hanging off them, in one path. Tags, types, and reuse targets go with the row because they are columns on it. Deleting a User is refused while any contribution names them as submitter, contributor, or risk owner, which matches how `InitiativeMembers` and `Tasks` already behave. The blob objects behind `ContributionAttachments` are not removed by the database. Deleting those is the service's job, exactly as with task comment attachments.

The root table

Fourteen columns, three of which carry a whole list each.

Contributions				one row per contribution, draft or submitted
COLUMN	TYPE	NULL	NOTES	
PK Id	int identity	no		
FK InitiativeId	int	no	Initiatives.Id, cascade. Step 1.	
FK SubmittedByUserId	int	no	Users.Id, restrict. Taken from the token, never from the request body.	
Title	nvarchar(200)	no	Step 3, required by the wizard's Next gate.	
Description	nvarchar(4000)	no	Step 3, also required by the gate.	
KeyTakeaway	nvarchar(500)	yes	One quotable sentence.	
Priority	nvarchar(20)	no	enum <code>ContributionPriority</code> , defaults to Medium.	
Status	nvarchar(20)	no	enum <code>ContributionStatus</code> , defaults to Draft. Step 8 sets it.	
Types	nvarchar(200)	no	JSON array of <code>ContributionType</code> names. Step 2. Dedupe in the service.	
Tags	nvarchar(1000)	no	JSON array of free text. Step 3. Trim and dedupe in the service.	
ReuseTargets	nvarchar(400)	no	JSON array of <code>ContributionReuseTarget</code> names. Step 7.	
SubmittedAt	datetime2	yes	Set once, when Status moves to Submitted. Null on drafts.	

COLUMN	TYPE	NULL	NOTES
CreatedAt	datetime2	no	UTC, via the existing value converter.
UpdatedAt	datetime2	yes	

Suggested indexes: `(InitiativeId, CreatedAt DESC)` for the feed,
`(SubmittedByUserId)` for "my contributions".

Not stored: the client type carries `initiativeName` and `workstream` alongside `initiativeId`. Both belong to the Initiative and should be projected into the response DTO, the way `ActivityResponseDto` already fills `UserDisplayName`. Copying them onto the row lets them drift the moment an Initiative is renamed.

People and evidence

Three tables that survive because each row carries more than a value.

ContributionContributors				step 6, submitter is row one
COLUMN	TYPE	NULL	NOTES	
PK Id	int identity	no		
FK UQ ContributionId	int	no	cascade	
FK UQ UserId	int	no	Users.Id, restrict	
ResponsibilityArea	nvarchar(255)	yes	same length as <code>InitiativeMember.ResponsibilityArea</code>	

COLUMN	TYPE	NULL	NOTES
IsPrimary	bit	no	the Primary / Supporting toggle, several rows may be primary
AddedAt	datetime2	no	

ContributionAttachments			metadata only, bytes live in blob storage
COLUMN	TYPE	NULL	NOTES
PK Id	int identity	no	
FK ContributionId	int	no	cascade
FileName	nvarchar(255)	no	what the person called it
BlobName	nvarchar(500)	no	key in the container, prefix <code>contributions/{id}</code>
ContentType	nvarchar(100)	no	
FileSize	bigint	no	bytes. 25 MB ceiling, same constant as task attachments.
CreatedAt	datetime2	no	

ContributionLinks			step 4 assets and step 5 links both land here
COLUMN	TYPE	NULL	NOTES
PK Id	int identity	no	
FK ContributionId	int	no	cascade

COLUMN	TYPE	NULL	NOTES
Source	nvarchar(20)	no	enum <code>ContributionLinkSource</code>
Url	nvarchar(2048)	no	validate the scheme is http or https before storing
Label	nvarchar(200)	yes	the wizard falls back to the URL when blank, do that in the UI not the column
Description	nvarchar(2000)	yes	absorbed from the dropped SupportingAssets table
CreatedAt	datetime2	no	

Conditional detail tables

Each uses `ContributionId` as both primary key and foreign key, a shared primary key. That gives a real 1:1 for free: the row cannot exist without its contribution, and there cannot be two of them. A contribution tagged with three types gets three of these rows, one per table.

ContributionMetrics				type: Business Metric
COLUMN	TYPE	NULL	NOTES	
PK FK ContributionId	int	no	cascade, shared primary key	
MetricName	nvarchar(150)	no		
Unit	nvarchar(50)	yes	users, %, hours	
PreviousValue	decimal(18,4)	yes	numeric, see decision 3	

COLUMN	TYPE	NULL	NOTES
CurrentValue	decimal(18,4)	yes	numeric, see decision 3
ReportingPeriod	nvarchar(50)	yes	free text, the wizard defaults it to "Q3 FY26"

ContributionRisks				type: Risk / Blocker
COLUMN	TYPE	NULL	NOTES	
PK FK ContributionId	int	no	cascade, shared primary key	
Description	nvarchar(2000)	no		
Severity	nvarchar(20)	no	enum <code>RiskSeverity</code> , defaults to Medium	
BusinessImpact	nvarchar(2000)	yes		
Mitigation	nvarchar(2000)	yes		
SupportNeeded	nvarchar(500)	yes		
FK OwnerUserId	int	yes	Users.Id, restrict. See decision 4.	
TargetResolutionDate	date	yes	see decision 5	

ContributionAiPractices				type: AI Best Practice
COLUMN	TYPE	NULL	NOTES	
PK FK ContributionId	int	no	cascade, shared primary key	

COLUMN	TYPE	NULL	NOTES
Tool	nvarchar(100)	no	free text, the wizard defaults it to "Copilot"
UseCase	nvarchar(2000)	yes	
Prompt	nvarchar(max)	yes	pasted verbatim so others can reuse it, no length ceiling worth guessing
TimeSavedHoursPerWeek	decimal(9,2)	yes	see decision 6
Recommendation	nvarchar(2000)	yes	

ContributionCustomerStories				type: Customer Story
COLUMN	TYPE	NULL	NOTES	
PK FK ContributionId	int	no	cascade, shared primary key	
CustomerName	nvarchar(200)	no		
Summary	nvarchar(4000)	yes		
Outcome	nvarchar(300)	yes	"43% faster onboarding"	
Quote	nvarchar(2000)	yes	what the Testimonials page will read	
BusinessValue	nvarchar(500)	yes		

New enums

Stored as strings with `HasConversion<string>()`, matching the five that already exist. Three of the six are element types of a JSON collection rather than scalar columns, which the probe confirmed stores member names, not integers.

ContributionType

ProgressUpdate · Deliverable · CustomerStory · BusinessMetric · Risk · Decision · AiBestPractice · Testimonial · SupportingAsset · SupportNeeded · Other

11 values, element type of `Contributions.Types`. Closed set: the wizard has an explicit "Other" card, so no free-text escape hatch is needed.

ContributionReuseTarget

ExecutiveBrief · LeadershipUpdate · Qbr · CustomerStoryLibrary · KnowledgeRepository · BestPractices · TeamNewsletter · VivaEngage

8 values, element type of `Contributions.ReuseTargets`, matching the REUSE_TARGETS array in the wizard.

ContributionLinkSource

SharePoint · Teams · Loop · OneDrive · ExternalUrl

5 values, scalar column on `ContributionLinks`, matching LINK_SOURCES.

ContributionPriority

Low · Medium · High · Critical

4 values, scalar. Deliberately not `InitiativePriority`, which has no Critical.

ContributionStatus

Draft · Submitted

2 values, scalar, both already in the client type. Room to add Archived later.

RiskSeverity

Low · Medium · High · Critical

4 values, scalar. Same members as ContributionPriority but a different concept, kept apart the way the codebase already keeps InitiativePriority and InitiativeTaskPriority apart.

Decisions worth making before the migration

Places where the wizard's current shape and a schema that will still be useful in a year disagree. Numbers 3 through 7 all involve changing a text input in the frontend.

1 The submitter is hardcoded

The wizard uses a `SUBMITTER` constant, "Nihar Pulluri". Backend takes the author from the token via `ICurrentUserService`, the way `ActivityService` and `TaskCommentService` do, and the wizard reads the real person from `useBackendUser`.

2 The initiative and people lists are mock data

Steps 1 and 6 read from `@/data/mock`. They need `useInitiatives` and the shared `UserPicker`, which already exist and are already wired to the API.

3 Metric values are strings

`previousValue` and `currentValue` are text inputs. Stored as `decimal(18,4)`, Analytics can compute the delta and chart it. Stored as text, it can only be displayed. The wizard needs numeric inputs.

4 The risk owner is a name, not a person

The field defaults to the submitter's display name and accepts anything. A `UserPicker` and an `OwnerUserId` make "risks I own" a query instead of a string match. Left nullable so an unassigned risk is still recordable.

5 Target resolution is free text

The placeholder suggests "End of Q3". A `date` makes overdue risks findable. If fuzzy periods genuinely matter more than sorting, say so and it stays `nvarchar(100)`.

6 Time saved is free text

"4 hrs / week" cannot be summed, and summing it across contributions is the obvious reason to capture it. A number plus a fixed "hours per week" label in the UI is the smaller change.

7 File size is a formatted string

The client stores "1.4 MB". Store `bigint` bytes and format on display, which is what `humanSize()` in the wizard already does and what `TaskCommentAttachment.FileSize` already holds.

8 The JSON columns give up two things

A child table enforced `UNIQUE(ContributionId, Tag)`. A JSON array does not, so nothing stops `["APAC", "APAC"]`. Dedupe in the service, the way `ActivityService.ResolveMentionsAsync` already does for mentions. Tag filters also cannot use an index: `OPENJSON` means a scan plus a parse per row. At a few thousand contributions that is milliseconds. Past roughly a hundred thousand it would need a persisted computed column, or promotion back to a table. That promotion is a straightforward migration, so this is not a one-way door.

9 Tags will fragment without a typeahead

Copilot, copilot, and co-pilot become three tags for one idea. Add `GET /api/contributions/tags?q=` over the distinct query in section 3, so the second person to type "cop" is offered what the first person created. The Initiative create form already does this against a hardcoded array at `initiatives-new.tsx:88`.

10 Should adding a contributor enrol them on the Initiative?

Assigning a task already adds the assignee to `InitiativeMembers` automatically. The same question applies here, and the answer should match so the Team tab stays trustworthy.

11 Uploads are simulated

Step 5's progress bar is a `setInterval` counting to 100. Real uploads hit the same ordering constraint that bit us on task comments: the row must exist before its attachments. Saving a draft first gives a `ContributionId` to upload against, which the wizard's Save draft button already fits.

12 The Decision type has no detail section

Its card promises "record a key decision and rationale" but step 4 has nothing for it. Either add a fifth conditional section and table, or accept that Description carries it. This is a UI gap, not a schema one, so it does not block the migration.

Build order

Same three steps as the last five tables, one migration for the whole set.

- **Step 1.** Six enums, eight entity classes with their `const` length limits, eight EF configurations including the three `PrimitiveCollection` calls, one migration. Verify with `has-pending-model-changes`, then update the database.
- **Step 2.** `ContributionsController` at `/api/initiatives/{id}/contributions` for the list and create, plus `/api/contributions/{id}` for read, update, and delete. Attachments follow the task pattern: `/api/contributions/{id}/attachments` to upload, and a download route. One create call carries the whole graph, since the wizard submits all eight steps at once.
- **Step 3.** Replace the mock arrays in the wizard, add `use-contributions.ts`, and point the Team Contributions page and the Initiative detail Contributions count at the API instead of `ContributionContext` state.

One thing to confirm before step 1: whether the `AddActivity` migration is applied on your database. It was still pending the last time we checked. Running `Update-Database` now will apply both if it is not.